[1차과제-c] 시나리오기획과 1차코드...
PDF

지난 시간에 1차 과제 했던거 기억하지? 이제 2차 과제를 진행해보자.

7s 동안 생각함 ›

응, 기억하고 있어요.
1차 때 만든 흐름은 이렇게였죠.

- 주제: **야구 타자 성적 관리 시스템**
- V1.0 기능: 타자 이니셜, 경기 수, 타수, 안타 수, 홈런 수, 지난 시즌 타율 입력
- 계산: 현재 타율, 지난 시즌과의 타율 차이
- 사용 문법: 변수, 자료형, `scanf`, `printf`, 사칙연산, `\t`, `%.3f` main
- README에는 2차를 **조건문을 이용한 조건 분기 적용** 단계로 남겨둔 상태였어요. README

2차 과제는 과제 안내 기준으로 **조건문을 활용한 데이터 분기 처리**가 핵심이에요. 성적 관리 프로그램에서는 평균 점수 구간으로 학점 A~F를 부여하고, 복합 조건으로 장학금 여부를 판단하는 식이었죠.

[1차과제-c] 시나리오기획과 1차코드작성

우리 야구 프로그램에서는 이렇게 확장하면 딱 맞아요.

2차 V2.0 추가 기능

1. 현재 타율에 따라 타자 등급 판정
2. 타율과 홈런 수를 함께 고려해 강타자 여부 판정

예를 들면:

- 타율 0.330 이상: S급 타자
- 타율 0.300 이상: A급 타자
- 타율 0.270 이상: B급 타자
- 타율 0.250 이상: C급 타자
- 그 외: D급 타자

그리고 복합 조건은:

- 타율 0.300 이상이고 홈런 30개 이상이면 "강타자"
- 아니면 "일반 타자"

이렇게 가면 2차 과제의 **if ~ else if ~ else**, **중첩 조건문**, **논리 연산자 &&**를 자연스럽게 넣을 수 있어요.

타율 등급을 판정할 때 조건 순서를 낮은 기준부터 작성했을 때와 높은 기준부터 작성했을 때의 차이가 뭐야?

좋은 질문이야 👍
이건 **if ~ else if 조건문의 동작 원리**를 이해하면 바로 풀려.

✅ 핵심 원리

`if ~ else if ~ else` 는 **위에서부터 순서대로 검사해서, 처음 참(True)인 조건 하나만 실행하고 끝나.**

즉:

```C
if (조건1)
{
}
else if (조건2)
{
}
```

```
else if (조건3)
{
}
```

👉 조건1이 참이면 밑에는 아예 안 봐.

🔥 그래서 순서가 엄청 중요해

네 타율 기준이:

- 0.330 이상 → S급
- 0.300 이상 → A급
- 0.270 이상 → B급
- 0.250 이상 → C급
- 그 외 → D급

❌ 낮은 기준부터 작성하면 문제 생김

```c
```

```c
if (current_avg >= 0.250)
{
    C급
}
else if (current_avg >= 0.270)
{
    B급
}
else if (current_avg >= 0.300)
{
    A급
}
else if (current_avg >= 0.330)
{
    S급
}
```

예를 들어 타율이:

```
0.340
```

이면?

맨 위 조건부터 검사:

맨 위 조건부터 검사:

```
0.340 >= 0.250   → 참
```

👉 바로 **C급** 출력하고 끝남 ❌
(S급이어야 하는데 망함)

✅ 높은 기준부터 작성해야 맞음

```C
if (current_avg >= 0.330)
{
    S급
}
else if (current_avg >= 0.300)
{
    A급
}
else if (current_avg >= 0.270)
{
    B급
}
else if (current_avg >= 0.250)
{
```

```
    C급
}
else
{
    D급
}
```

타율 0.340이면:

- 0.340 >= 0.330 → 참

👉 **S급 출력** (정상)

🎯 왜 이런 일이 생기냐?

조건들이 서로 포함 관계라서 그래.

예:

```
0.340 은
0.330 이상 ✓
0.300 이상 ✓
0.270 이상 ✓
0.250 이상 ✓
```

즉 여러 조건을 동시에 만족해.

그래서 **가장 엄격한 조건부터 먼저 검사해야 함**

✅ 한 줄 공식

👉 **점수/등급 판정은 높은 기준 → 낮은 기준 순서**

예:

- 학점 A,B,C,D,F
- 온도 위험 단계
- 회원 등급
- 야구 타율 등급

전부 동일해.

🔥 교수님이 이걸 보는 이유

2차 과제 핵심이 조건문이니까:

- 조건 순서 이해했는지
- else if 구조 이해했는지
- 논리적으로 분기했는지

이걸 보는 거야.

🎯 한 줄 정리

👉 낮은 기준부터 쓰면 **높은 등급이 낮은 등급에 먼저 걸려서 오답**
👉 높은 기준부터 써야 정확한 판정 가능

강타자 여부를 판정할 때 논리 연산자 &&을 사용해서 타율과 홈런 수를 고려하는 조건을 한번에 썼을 때의 장점이 뭐야?

좋은 질문이야 👍
이건 **왜 `&&`를 쓰는지**, 그리고 **논리 연산자가 왜 필요한지**를 이해하는 핵심이야.

✅ 먼저 `&&` 뜻

```c
조건1 && 조건2
```

👉 **조건1도 참이고 조건2도 참일 때만 참(True)**

즉 둘 다 만족해야 함.

🔥 우리 코드 예시

```c
```

```c
if (current_avg >= 0.300 && home_run >= 30)
{
    printf("강타자입니다.");
}
```

뜻은:

- 타율 0.300 이상
- 홈런 30개 이상

👉 **둘 다 만족하면 강타자**

✅ 한번에 쓰는 장점 1. 조건이 명확함

```c
타율 >= 0.300 && 홈런 >= 30
```

이렇게 보면 바로 알 수 있어.

👉 "강타자의 기준이 뭔지 한 줄에 보임"

❌ 따로 쓰면 복잡해짐

```c
if (current_avg >= 0.300)
{
    if (home_run >= 30)
    {
        강타자
    }
}
```

기능은 같지만

- 줄 수 많아짐
- 들여쓰기 늘어남
- 보기 복잡함

✅ 장점 2. 코드가 짧고 깔끔함

```c
if (조건1 && 조건2)
```

한 줄이면 끝.

👉 가독성 좋음
👉 실수 줄어듦

✅ 장점 3. 기준을 동시에 만족해야 함

예를 들어:

타율	홈런	결과
0.320	35	강타자
0.320	20	일반 타자
0.280	35	일반 타자

👉 하나만 높다고 강타자가 아님
👉 **둘 다 충족해야 함**

이걸 정확하게 표현하는 게 `&&`

🔥 중첩 조건문과 비교

중첩 조건문

```c
if (타율 >= 0.300)
{
    if (홈런 >= 30)
```

👉 단계적으로 검사

`&&`

```c
if (타율 >= 0.300 && 홈런 >= 30)
```

👉 한 번에 검사

🎯 한 줄 정리

👉 `&&`를 쓰면 **여러 조건을 동시에 만족하는 기준을 고 명확하게 표현할 수 있다.**